

1. 💡 Arquitectura y Plan de Acción Estratégico

El desafío principal es estructurar el código para que la lógica de negocio sea **reutilizable** en el Backend (Node/Express), el Backoffice Web (React), la App Móvil (React Native/Expo) y el Backoffice Interno. El patrón de diseño ideal para esto dentro de tu monorepo es el de **Capas (Layered Architecture)**, con el corazón de la lógica en la librería **shared**.

🔗 Estructura del Monorepo con Workspaces

Tu estructura actual es buena, pero podemos expandirla para dar cabida a las nuevas plataformas y la lógica compartida de POO.

Carpeta	Plataforma/Capa	Tecnologías	Propósito
backend/	Capa de Infraestructura (API)	Express/TS	Recibe peticiones HTTP, delega la lógica al <i>Domain</i> y gestiona la DB. Microservicios.
frontend/	Capa de Presentación (Web - Empresa)	React/Vite/TS	Interfaz para la plataforma 1. Consume los <i>Microservicios</i> del Backend.
app-mobile/	Capa de Presentación (Móvil - Usuario)	React Native/Expo/TS	Interfaz para la plataforma 2. Consume los <i>Microservicios</i> del Backend.
backoffice-admin/	Capa de Presentación (Web - Interna)	React/Vite/TS	Interfaz para la plataforma 3. Consume los <i>Microservicios</i> del Backend.
shared/	CAPA DE DOMINIO/LÓGICA	TypeScript	Contiene las clases, interfaces y lógica de negocio reutilizable (el modelado de POO).
docs/	Documentación		

Exportar a Hojas de cálculo

Acción Inmediata:

1. Crear las nuevas carpetas: app-mobile/ y backoffice-admin/.

2. Asegurarse de que `shared/` sea un *workspace* con un `package.json` para ser importado como dependencia local en `backend/`, `frontend/`, `app-mobile/`, y `backoffice-admin/`.
-

2. □ Modelado Orientado a Objetos (POO) con Clases e Interfaces

El modelado POO se basará en gran medida en tu esquema BDD.sql. La idea es que cada tabla principal tenga una **Clase de Dominio** (la entidad real con métodos) y una **Interfaz** (la estructura de datos). Esto se desarrollará íntegramente en la carpeta **shared/src/**.

Ejemplo: Módulo de Gestión de Eventos (Eventos)

Interfaz (Contrato de Datos)

Define la estructura de datos.

TypeScript

```
// shared/src/events/interfaces/IEvent.ts
```

```
export interface IEvent {  
  id: string; // UUID  
  company_id: string; // UUID  
  title: string;  
  description: string;  
  category_id: number;  
  location_id?: string;  
  starts_at: Date;  
  ends_at?: Date;  
  timezone?: string;  
  capacity?: number;  
  attendee_count: number;  
  visibility: 'public' | 'unlisted' | 'private';  
  is_published: boolean;  
  created_by: string;
```

```
}
```

Clase de Dominio (Entidad con Lógica)

Representa el objeto en el sistema con sus propiedades y **métodos de negocio**.

TypeScript

```
// shared/src/events/domain/Event.ts
```

```
import { IEvent } from '../interfaces/IEvent';
```

```
export class Event implements IEvent {
```

```
  // Propiedades (implementa IEvent)
```

```
  public id: string;
```

```
  public company_id: string;
```

```
  public title: string;
```

```
  public description: string;
```

```
  // ... resto de propiedades
```

```
  constructor(data: IEvent) {
```

```
    Object.assign(this, data);
```

```
  }
```

```
  /**
```

```
   * Método de negocio: Valida si se pueden añadir más asistentes.
```

```
   * @param count El número de nuevos asistentes.
```

```
   */
```

```
  public canAddAttendees(count: number = 1): boolean {
```

```
    if (this.capacity === null || this.capacity === undefined) {
```

```
      return true; // No hay límite de capacidad
```

```
    }
```

```

        return (this.attendee_count + count) <= this.capacity;
    }

    /**
     * Método de negocio: Publica el evento, cambiando su estado.
     */
    public publish(): void {
        if (this.is_published) {
            throw new Error('El evento ya está publicado.');
```

```
        }
```

```
        // Lógica para revisar si todos los campos requeridos están completos
```

```
        if (!this.title || !this.starts_at) {
```

```
            throw new Error('No se puede publicar sin campos esenciales.');
```

```
        }
```

```
        this.is_published = true;
```

```
    }
```

```
}
```

Otros Modelos Clave

Debes replicar este patrón para las siguientes tablas clave:

Módulo/Tabla	Interfaz (I...)	Clase (...)	Lógica de Negocio Típica (Métodos)
users	IUser	User	hashPassword(newPass), checkPermissions(), isEmailVerified()
companies	ICompany	Company	isSubscriptionActive(), getPlanFeatures(), calculateMonthlyRevenue()

Módulo/Tabla	Interfaz (I...)	Clase (...)	Lógica de Negocio Típica (Métodos)
adhesiones	IAdhesion	Adhesion	canBeApproved(), changeStatus(newStatus), uploadDocument(file)
reviews	IReview	Review	moderate(action), calculateRatingTrend()
event_registrations	IEventRegistration	Registration	cancel(), confirm(), checkIn(user), getNoShowRate()

Exportar a Hojas de cálculo

3. 🌐 Implementación en las Capas de la Aplicación

Una vez que la lógica de dominio (Clases y Métodos POO) esté en **shared/**, la implementación de los microservicios y la interfaz de usuario será mucho más simple.

3.1. ⚙️ Capa de Microservicios (Backend)

En **backend/**, los microservicios usarán clases para manejar la lógica de negocio antes de interactuar con la DB. Esto sigue el patrón de diseño de **Servicios de Aplicación**.

- **Tecnología:** Node.js + Express + TypeScript.
- **Patrón: Repository Pattern** para la conexión con Supabase.

Flujo del Microservicio de Creación de Eventos

1. **Endpoint (Controller):** Recibe el request HTTP.
2. **Servicio:** Recibe los datos, **valida** la información, y crea una instancia de la clase de dominio.

TypeScript

```
// backend/src/events/EventService.ts
```

```
import { Event, IEvent } from 'shared/events'; // Importa la clase de dominio
```

```
// ... Importa el EventRepository
```

```

class EventService {
  constructor(private eventRepository: EventRepository) {}

  public async createEvent(eventData: IEvent): Promise<IEvent> {
    // 1. Lógica de Dominio: Crea la instancia de Event
    const newEvent = new Event(eventData);

    // 2. Ejecuta un método de la clase de dominio para validación/preparación
    // newEvent.validateCreationData(); // Método de la clase Event

    // 3. Ejecuta la lógica de persistencia (va a la DB)
    const savedEvent = await this.eventRepository.save(newEvent);

    return savedEvent;
  }
}

```

3.2. 🖥️ Capas de Presentación (Frontend, App Mobile, Backoffice Admin)

Las interfaces de usuario también pueden importar las clases de **shared/** para validar datos localmente y manipular información antes de enviarla a los microservicios (por ejemplo, al mostrar un evento).

- **Tecnología:** React, React Native, TypeScript.

Uso en el Frontend

TypeScript

```
// frontend/src/pages/EventDetails.tsx
```

```
import { IEvent, Event } from 'shared/events';
```

```
import React, { useState, useEffect } from 'react';
```

```

// Supongamos que eventData viene de una API
const eventData: IEvent = { /* ... datos del evento ... */ };

const EventDetails = () => {
  const [eventInstance, setEventInstance] = useState<Event | null>(null);

  useEffect(() => {
    // Aquí se crea la instancia de la clase para usar sus métodos
    setEventInstance(new Event(eventData));
  }, [eventData]);

  const handleRegistration = () => {
    if (eventInstance && eventInstance.canAddAttendees()) {
      // Si la lógica de dominio local es OK, llama al microservicio
      alert("Listo para registrar. Llamando a API...");
      // API.register(eventInstance.id);
    } else {
      alert("El evento está lleno o no cumple con las reglas.");
    }
  };

  return (
    <div>
      <h2>{eventInstance?.title}</h2>
      <button onClick={handleRegistration}
        disabled={!eventInstance?.is_published}>
        Inscribirse
    </div>
  );
}

```

```

</button>

{/* ... */}

</div>

);

};

```

4. 🗄️ Gestión de Base de Datos y Supabase

El esquema BDD.sql es avanzado y bien pensado. **No necesitas cambiarlo**, ya que es la base para el modelado POO.

- **Microservicios (Backend):** Usarán el **patrón Repository** para interactuar con la DB. Esto aísla la lógica de acceso a datos de la lógica de negocio.
 - **DAO/Repository:** Contiene métodos como `findById(id)`, `save(event: Event)`, `delete(id)`.
 - **Implementación:** Usará la API de Supabase o una librería como `pg` directamente para ejecutar SQL o usar sus funciones (ej. `SELECT * FROM get_upcoming_events(...)`).
 - **Vistas y Funciones SQL:** Las vistas (`v_events_full`, `v_event_statistics`) y funciones SQL (`is_event_full`, `get_upcoming_events`) son esenciales para la **performance** y deben ser llamadas por el Repository Pattern para evitar lógica compleja en el Backend.
-

5. ✅ Plan de Acción Detallado (Próximos Pasos)

Basado en tu README.md y la nueva arquitectura:

Paso	Módulo	Acciones Clave	Ubicación del Archivo
Fase 1: Preparación del Monorepo	Configuración	1. Configurar npm workspaces en el <code>package.json</code> principal.	<code>package.json (root)</code> , <code>shared/</code>
		2. Crear las carpetas <code>app-mobile/</code> y <code>backoffice-admin/</code> .	

Paso	Módulo	Acciones Clave	Ubicación del Archivo
Fase 2: Implementar Dominio POO	Autenticación , Eventos	3. Configurar la librería shared/ y enlazarla como dependencia local.	
		1. Implementar IUser, User con métodos de autenticación (checkPassword, generateToken). 2. Implementar IEvent, Event con métodos de validación (canAddAttendees, publish). 3. Implementar ICompany, Company para la gestión de roles y planes.	shared/src/users/, shared/src/events/, shared/src/companies/
Fase 3: Implementar Microservicio Base	Autenticación	1. Implementar el Repository para users que interactúe con Supabase. 2. Crear el Microservicio de Autenticación (AuthService) que use las Clases de Dominio. 3. Configurar las rutas de Express para Login/Logout.	backend/src/auth/

Paso	Módulo	Acciones Clave	Ubicación del Archivo
Fase 4: Dashboard y Módulos	Dashboard, Eventos	1. Crear el Microservicio de Eventos (EventService) usando el patrón Repository. 2. Crear el Dashboard principal en frontend/ y backoffice-admin/ que consuma métricas de las vistas SQL (ej. v_event_statistics) a través de la API.	backend/src/events/, frontend/src/pages/Dashboard.ts x

Exportar a Hojas de cálculo

¿Te gustaría que profundice en la estructura de clases del módulo de **Autenticación y Autorización** o del módulo de **Gestión de Reservas**?